

Да, это уточнение **существенно меняет архитектуру проекта**, и я теперь зафиксировал бы именно эту цель: **не ЕС и не мировой рынок, а четыре страны ЕАЭС — Беларусь, Россия, Казахстан и Армения.**

При этом в присланном рассуждении есть несколько важных упрощений, которые я бы не переносил в ТЗ как готовые факты.

Главное исправление

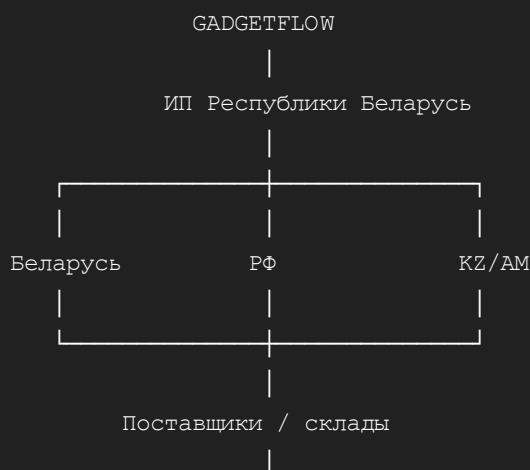
ЕАЭС ≠ «для интернет-магазина всё просто и таможни нет».

Для товаров, уже находящихся в свободном обращении в ЕАЭС, действительно нет обычной внутренней таможенной границы между государствами-членами в том смысле, как при экспорте в третью страну. Но **налоговое оформление, подтверждение операций, документы, НДС и национальные правила дистанционной торговли никуда не исчезают**. ЕЭК отдельно регулирует косвенные налоги во взаимной торговле, а белорусское МНС прямо указывает, что при операциях с государствами ЕАЭС применяются правила Протокола по косвенным налогам. 🚩 Евразийская ... +1

И ещё важнее: Беларусь сама признаёт дистанционную продажу товара с доставкой за пределы РБ отдельной ситуацией — при доставке покупателю за пределами Беларуси место реализации для целей белорусского НДС не признаётся территорией РБ. 🇧🇪 Налоги...

То есть мы действительно должны проектировать **трансграничный e-commerce**, а не просто «белорусский сайт с четырьмя переключателями стран».

Я бы теперь окончательно зафиксировал модель GadgetFlow



Fulfillment

|

Клиент

Причём страна клиента и страна склада поставщика должны быть независимыми параметрами.

Это критично.

Например:

ИП РБ
↓
поставщик РБ
↓
покупатель РФ

и

ИП РБ
↓
поставщик РФ
↓
покупатель РФ

— это две разные экономические и налоговые модели.

Во втором случае мы уже не можем просто сказать «поставщик отправляет товар покупателю». Нужно отдельно разбирать движение товара, право собственности, расчёты и налоговые последствия.

Поэтому я предлагаю не одну модель поставщика, а две

Модель А — приоритетная

Поставщик находится в Беларуси.

Поставщик РБ
↓
GadgetFlow / ИП РБ
↓
покупатель РБ / РФ / KZ / AM

Это наиболее чистая стартовая архитектура.

Модель В — последующая

Поставщик находится непосредственно в стране покупателя.

Например:

```
Поставщик РФ
↓
Покупатель РФ
```

при том что договор с клиентом заключён с ИП РБ.

Вот здесь уже возникает отдельный пласт регулирования, и такой сценарий я бы **не** включал в первую production-версию без отдельного налогового и юридического заключения.

И самое важное изменение в техническом задании

Я бы не делал

```
country = BY
currency = BYN
```

и потом через полгода «добавим страны».

Сразу должна существовать полноценная сущность:

```
Country
Currency
TaxProfile
PaymentProfile
DeliveryProfile
LegalProfile
ConsumerProtectionProfile
```

Например:

```
BY
RU
KZ
AM
```

и:

```
BYN
RUB
KZT
AMD
```

Но включение страны в интерфейс ≠ разрешение продавать в этой стране.

У страны должен быть production-флаг:

```
enabled
paymentEnabled
deliveryEnabled
ordersEnabled
```

Поэтому можно иметь:

```
RU:
  enabled: true
  paymentEnabled: false
  deliveryEnabled: false
  ordersEnabled: false
```

Сайт уже готов, но Россия не активирована.

Это как раз позволит нам не переделывать проект.

Цены тоже надо сделать правильно

Не рекомендую:

цена BYN × сегодняшний курс = цена RUB

На реальном магазине так лучше не делать.

Должна быть базовая цена:

```
basePrice
baseCurrency
```

и отдельно:

```
marketPrice
marketCurrency
```

Потому что для российского рынка цена может учитывать:

- курс;
- комиссию платежа;
- конвертацию;
- стоимость доставки;
- возвратный риск;
- налоговую нагрузку;
- локальную конкурентную цену.

Например:

```
Product
├─ price.BY = 149.00
├─ price.RU = 4290
├─ price.KZ = 25990
└─ price.AM = 18900
```

а не постоянное вычисление на frontend.

Checkout теперь должен быть мультистрановым

Например:

```
Country
  ↓
City / Region
  ↓
Delivery method
  ↓
Currency
  ↓
Payment methods
  ↓
Tax rules
  ↓
Final price
  ↓
Order
```

А сервер должен сам определить:

```
market
currency
paymentProvider
deliveryProvider
```

То есть клиентский `checkout-form.tsx` не должен сам решать юридическую модель заказа.

Что я категорически не рекомендую делать сейчас

Не нужно одновременно подключать:

bePaid + 2Checkout + российский gateway + криптовалюту + несколько касс.

Это будет технический монстр.

Начинаем с:

Беларусь

```
BYN
↓
локальный payment gateway
↓
белорусская кассовая схема
↓
доставка РБ
```

Затем отдельным country module:

Россия

```
RUB
↓
допустимый способ оплаты
↓
транграничная модель
↓
delivery RU
↓
отдельная юридическая/налоговая логика
```

Потом KZ.

Потом AM.

Есть ещё одна важная поправка к твоей таблице

Фраза:

«Валютный контроль — Да, нужны документы на поступления в €/\$/RUB и т.д.»

слишком категорична.

Расчёты с иностранными покупателями действительно надо рассматривать как часть валютного регулирования и банковского compliance, но **не всякий платёж автоматически означает одинаковую процедуру валютного контроля**.

Конкретный порядок зависит от характера операции, договора, банка и применимого валютного законодательства.

Поэтому в архитектуре мы не будем кодировать предположение:

```
foreignCurrency = true → always manual currency control
```

Мы заложим:

```
foreignSettlement  
bankComplianceRequired  
contractReference  
settlementCurrency  
paymentProvider
```

а конкретный compliance-процесс определим до активации соответствующего рынка.

И ещё: оплата российской картой ≠ автоматически bePaid

Вот это я бы вообще убрал из технического задания.

Нельзя исходить из:

«bePaid принимает российскую карту → значит российский покупатель может нормально купить».

Нужно проверить одновременно:

1. поддерживаемый способ оплаты;
2. страну эмитента карты;
3. валюту;
4. правила конкретного эквайера;
5. возможность зачисления ИП РБ;
6. требования банка;
7. санкционные/комплаенс-ограничения;

8. возвраты;
9. кассовую модель;
10. подтверждение операции.

Это нужно проверять не по документации 2024–2025 годов, а непосредственно перед запуском в 2026-м.

И здесь есть очень интересная вещь в официальных данных МНС

Белорусская налоговая прямо указывает, что при электронной дистанционной продаже белорусским продавцом с доставкой за пределами РБ место реализации товара для целей белорусского НДС не является территорией Беларуси.

 Налоговый с...

То есть твой сценарий:

ИП РБ
↓
товар
↓
покупатель РФ

нельзя просто реализовать как:

checkout → RUB → payment → receipt

Нам необходимо будет определить налоговую модель трансграничного B2C заказа до подключения реальной оплаты.

Это один из самых важных выводов после твоего уточнения.

Поэтому мой окончательный план для GadgetFlow теперь такой

Уровень 1 — бизнес

ИП РБ

↓

розничный интернет-магазин



договоры с поставщиками



поставщики могут физически хранить и отправлять товар



GadgetFlow остаётся продавцом перед своим клиентом, если именно такая договорная модель будет выбрана.

Уровень 2 — рынки

Сразу в архитектуре:

BY — подготовить и запустить

RU — подготовить, но включить после legal/payment review

KZ — подготовить, но включить после legal/payment review

AM — подготовить, но включить после legal/payment review

Уровень 3 — валюты

BYN

RUB

KZT

AMD

с нормальной backend-моделью денег, а не `float` и не пересчётом на фронтенде.

Я бы сразу закладывал integer minor units:

14900 BYN minor units

429000 RUB minor units

2599000 KZT minor units

где это соответствует правилам конкретной валюты.

Уровень 4 — поставщики

Каждый товар должен иметь:

```
supplierId
supplierSku
warehouseCountry
warehouseRegion
stock
purchasePrice
currency
deliverySla
warranty
complianceDocuments
```

И именно здесь мы сможем потом сделать настоящий Shopify-подобный dropshipping engine.

Что особенно важно для электроники

Для каждого SKU я бы сделал обязательную проверку:

```
EAEU compliance
Certificate / Declaration
Manufacturer
Importer
Country of origin
Warranty
Marking
EAN
HS / TN VED code
```

Потому что продавать просто «красивый китайский гаджет» без понимания документов для ЕАЭС — плохая идея.

Именно поставщик должен предоставить подтверждение, что товар легально введён в оборот в соответствующем рынке, а мы должны хранить эти сведения в системе.

А вот Китай всё-таки остаётся в нашей архитектуре — но не как стартовый канал

Вот так:

```
China
↓
supplier/importer
```

```
↓
Belarus warehouse
↓
GadgetFlow
↓
EAEU customer
```

А не:

```
China
↓
GadgetFlow
↓
customer
```

Это позволит тебе заниматься настоящим dropshipping, не превращая GadgetFlow в импортёра каждого отдельного гаджета.

Итоговое решение

Я бы теперь официально зафиксировал концепцию GadgetFlow следующим образом:

GadgetFlow — мультистрановой интернет-магазин товаров электроники и умных гаджетов, управляемый ИП Республики Беларусь. Архитектура с первого дня рассчитана на рынки Беларуси, России, Казахстана и Армении, валюты BYN/RUB/KZT/AMD, несколько поставщиков, различные складские страны, трансграничную доставку, разные платёжные профили и отдельные юридико-налоговые правила каждого рынка. При этом первым production-рынком становится Беларусь, а остальные рынки технически подготавливаются, но активируются только после отдельной проверки платёжной, налоговой, кассовой и потребительской модели.

Это я считаю **намного более правильным фундаментом**, чем либо «только Беларусь», либо попытка сразу открыть четыре страны.

И ещё один важный момент: начиная с 2026 года белорусское налоговое регулирование отдельно учитывает электронную дистанционную торговлю, а МНС уже ведёт соответствующие специализированные разъяснения и сервисы.

 Налоговый сайт +1

Следовательно, дальше нам нужен не набор разрозненных правок сайта, а единый `GadgetFlow EAEU Architecture & Legal Specification v1.0`. В него я бы включил одновременно бизнес-модель, юрмодель, страны, валюты, налоговые точки, поставщиков, БД, CMS, checkout, payment abstraction, кассу, delivery engine,

returns, compliance товаров, SEO и пошаговый порядок выхода BY → RU → KZ → AM.

Это уже будет тот документ, по которому можно давать команды твоему агенту и **не переделывать архитектуру через каждые две недели.**